

Input Validation Vulnerabilities

La validación de entrada de datos (Input Validation) es una práctica fundamental en la **ciberseguridad web**. El objetivo es asegurarse de que solo los datos esperados, correctos y seguros sean procesados por una aplicación. Si no se realiza una validación adecuada, una aplicación web podría convertirse en un punto de entrada para ataques maliciosos, lo que podría comprometer la seguridad de la información y el funcionamiento del sistema.

1. ¿Qué es la validación de entrada de datos y por qué es tan importante?

La validación de entrada es el proceso de verificar que los datos que recibe una aplicación web, provenientes de una fuente externa (como un usuario), son válidos y cumplen con las reglas y el formato esperado.

Es crucial por varias razones:

- **Protección contra ataques:** Previene vulnerabilidades como la **inyección SQL (SQL Injection)** y el **Cross-Site Scripting (XSS)**. Al validar los datos, se pueden detectar y rechazar peticiones maliciosas antes de que lleguen a ser procesadas, evitando que un atacante inyecte código o comandos dañinos.
- **Integridad de los datos:** Garantiza que la información almacenada o utilizada por la aplicación sea precisa y no cause errores. Si un usuario introduce un número donde se espera un texto, la aplicación podría fallar.
- **Funcionamiento correcto:** Evita que datos inesperados o malformados causen fallos o un comportamiento impredecible en los componentes de la aplicación.

- **Detección temprana de amenazas:** Al validar las entradas lo antes posible, se pueden identificar intentos de ataque en las primeras etapas, lo que facilita la respuesta y la mitigación de riesgos.
-

2. ¿Dónde y cuándo se debe realizar la validación?

La validación debe ocurrir en dos puntos clave para ofrecer la máxima seguridad:

a) Validación en el lado del cliente (Navegador)

Esta validación se realiza en el navegador del usuario antes de que los datos sean enviados al servidor. Se implementa con lenguajes como **JavaScript** y sirve principalmente para mejorar la experiencia de usuario. Por ejemplo, puede mostrar un mensaje de error inmediato si el usuario intenta ingresar letras en un campo de solo números.

- **Ventajas:** Es rápida, reduce la carga del servidor y guía al usuario para que rellene los formularios correctamente.
- **Inconvenientes: No es segura.** Un atacante puede fácilmente saltarse esta validación utilizando herramientas como un **proxy web** (por ejemplo, Burp Suite) que interceptan y modifican las peticiones HTTP antes de que lleguen al servidor. Por esta razón, la validación del lado del cliente nunca debe ser la única medida de seguridad.

b) Validación en el lado del servidor

Esta validación es la más crítica para la seguridad. Se lleva a cabo en el servidor web una vez que los datos han sido recibidos. El servidor revisa cada dato de una petición para asegurar que cumple con todas las reglas de validación.

- **Ventajas:** Es **obligatoria y segura**. No importa cómo un atacante manipule la petición en el cliente, el servidor siempre realizará su propia verificación.
 - **Inconvenientes:** Puede ser más lenta que la validación en el cliente, ya que implica una comunicación completa entre el navegador y el servidor.
-

3. ¿Cómo se implementa la validación de entrada?

Existen varias técnicas para implementar una validación de entrada efectiva. A continuación, se explican las más comunes y recomendadas:

- **Validadores de tipo de datos:** Aseguran que el dato recibido corresponde al tipo esperado (por ejemplo, un número entero, una cadena de texto, un booleano, etc.). Muchos frameworks web modernos (como Django o Spring) incluyen validadores nativos.
- **Conversión de tipo (Type Conversion):** En algunos casos, se puede convertir el dato de entrada al tipo esperado para forzar su formato. Por ejemplo, si se espera un número entero, el dato de entrada se puede convertir a `integer` para descartar cualquier carácter no numérico.
- **Rango de valores:** Se comprueba que el valor de un dato numérico o de fecha se encuentra dentro de un rango permitido. Por ejemplo, una fecha de nacimiento debe corresponder a una persona mayor de 18 años, o una cantidad de un producto debe estar entre 1 y 100.
- **Expresiones regulares (Regular Expressions):** Son una herramienta muy potente para validar formatos específicos. Permiten definir patrones que una cadena de texto debe seguir. Son ideales para validar:
 - **Correos electrónicos:**
`^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`
 - **Códigos postales:** `^\d{5}$` (en España, 5 dígitos)
 - **Números de teléfono:** `^\+\d{1,3} \d{9}$` (por ejemplo, para números de 9 dígitos con prefijo internacional)
- **Listas negras (Denylists) vs. Listas blancas (Allowlists)**
 - **Listas Negras:** Definen una lista de caracteres o patrones que **no están permitidos** (por ejemplo, `script`, `alert`, comillas simples `'`). Esta técnica **no es recomendable** porque un atacante puede encontrar fácilmente una manera de evadirla utilizando variaciones o codificaciones de caracteres. Es imposible prever y bloquear todos los posibles ataques.

- **Listas Blancas:** Definen una lista de caracteres o formatos que **sí están permitidos** y se consideran seguros. Cualquier cosa que no esté en la lista blanca es automáticamente rechazada. Esta es la **técnica más segura y recomendada** para la validación de entrada, ya que se centra en lo que es válido en lugar de lo que es malicioso.
 - **Normalización (Normalization):** Proceso de convertir los datos de entrada a un formato estándar o "canónico" para su posterior procesamiento. Esto es especialmente útil para campos de texto libre (como comentarios), donde una lista blanca estricta no es viable. La normalización asegura que el texto recibido sea tratado solo como texto plano, neutralizando cualquier intento de inyección de código. Por ejemplo, se podría convertir `<script>` a `<script>` para que el navegador lo muestre como texto en lugar de ejecutarlo.
-

4. Resumen y conclusiones

La validación de entrada de datos es un pilar fundamental en la seguridad web. Su implementación correcta **previene ataques de inyección** como **SQL Injection** y **XSS**. Es crucial realizarla tanto en el cliente (para una mejor experiencia de usuario) como, obligatoriamente, en el servidor (para una seguridad robusta).

La mejor estrategia para implementarla es el uso de **listas blancas (allowlists)**, que definen qué datos son aceptables. Complementar esta técnica con expresiones regulares, validación de tipos y rangos, y la normalización de datos asegura una defensa sólida contra las vulnerabilidades más comunes.

XSS: Cross Site Scripting

En el ámbito de la **ciberseguridad web**, una de las vulnerabilidades más críticas y comunes es el **Cross-Site Scripting (XSS)**. Se clasifica dentro de las vulnerabilidades de tipo **inyección** y se ha mantenido consistentemente en los primeros puestos del **OWASP Top 10**. A diferencia de otros ataques que se ejecutan en el servidor, el XSS se activa en el **navegador de la víctima**.

1. ¿Qué es el Cross-Site Scripting (XSS)?

El XSS es una vulnerabilidad que permite a un atacante inyectar código malicioso (generalmente **JavaScript**) en una página web que será visto por otros usuarios. El navegador de la víctima, al no distinguir entre el código legítimo de la página y el inyectado, lo ejecuta.

El objetivo de un ataque XSS puede ser variado, desde algo tan simple como mostrar un mensaje de alerta para demostrar la vulnerabilidad hasta acciones más maliciosas, como:

- **Robo de cookies:** El atacante puede robar las cookies de la sesión del usuario para suplantar su identidad.
 - **Redirección a sitios maliciosos:** Redirigir al usuario a una página de **phishing** para robar sus credenciales.
 - **Manipulación de la interfaz:** Alterar el contenido de la página para que el usuario, sin saberlo, envíe información sensible a un servidor del atacante.
-

2. Tipos de Cross-Site Scripting

Existen tres tipos principales de XSS, clasificados según cómo se inyecta y se propaga el **payload**:

a) XSS Reflejado (**Reflected XSS**)

Este tipo de ataque ocurre cuando el **payload** (el código malicioso) se envía directamente en la URL de una petición HTTP (normalmente **GET**). El servidor web "refleja" el **payload** en la respuesta HTML sin una validación adecuada, lo que provoca que se ejecute en el navegador del usuario.

Para que un ataque de XSS reflejado tenga éxito, el atacante debe recurrir a la **ingeniería social**. Debe engañar a la víctima para que haga clic en un enlace que ya contiene el **payload** malicioso.

Ejemplo práctico de XSS Reflejado:

Imagina que una página de búsqueda vulnerable tiene la siguiente URL:

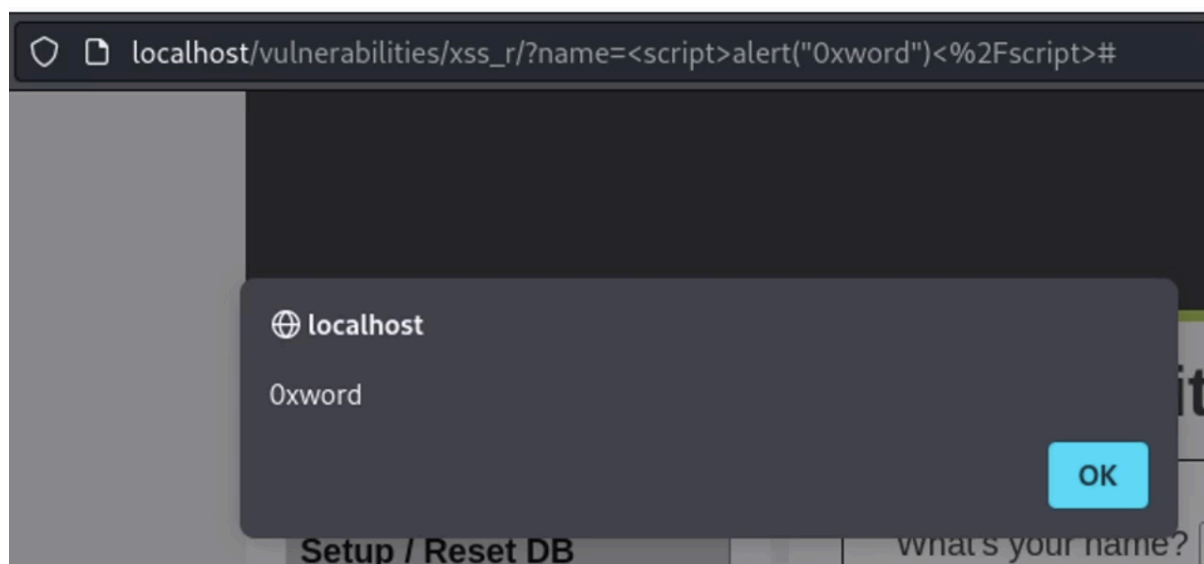
<http://ejemplo.com/buscar?q=tucadena>

Un atacante podría construir una URL maliciosa de la siguiente manera:

[http://ejemplo.com/buscar?q=<script>alert\('Ataque XSS'\)</script>](http://ejemplo.com/buscar?q=<script>alert('Ataque XSS')</script>)

Si la víctima hace clic en este enlace y la página no valida la entrada, el navegador de la víctima ejecuta el **script** y muestra una ventana emergente.

Análisis de un caso real:



La imagen de arriba muestra un ejemplo claro de un ataque XSS reflejado.

Analicemos la URL y su resultado:

URL inyectada:

```
localhost/vulnerabilities/xss_r/?name=<script>alert("0xword")<%2Fscript>#
```

- `localhost/vulnerabilities/xss_r/`: Es la dirección base de la página vulnerable.
- `?name=`: Es el parámetro que el atacante está inyectando.
- `<script>alert("0xword")<%2Fscript>`: Este es el **payload** de JavaScript. El `<%2Fscript>` es la versión URL-codificada de `</script>`, lo que hace que el `script` se interprete correctamente incluso en la URL. El `alert("0xword")` es un comando que le dice al navegador que muestre una ventana de alerta con el texto "0xword".
- `#`: El símbolo de almohadilla a menudo marca una "ancla" en la página, que el servidor ignora. En este caso, simplemente asegura que el navegador no envíe esa parte de la URL al servidor, lo que a veces puede ayudar a evadir ciertos filtros.

Cuando se carga esta URL, el navegador interpreta el `script` inyectado y muestra una ventana emergente, como se ve en la imagen, con el mensaje "0xword". Esta simple acción **demuestra la existencia de la vulnerabilidad**.

b) XSS Almacenado (Stored XSS)

Este es el tipo de XSS más peligroso. El **payload** malicioso se almacena de forma persistente en el servidor de la aplicación (por ejemplo, en una base de datos) y se entrega a los usuarios cada vez que acceden a la página vulnerable.

- **Ejemplo:** Un atacante deja un comentario en un blog que contiene un `script` malicioso. Cualquier usuario que visite la página de comentarios ejecutará el `script` sin saberlo. El atacante no necesita engañar a la víctima con un enlace, ya que la vulnerabilidad se activa de manera automática.

c) XSS Basado en DOM (DOM-based XSS)

En este caso, la vulnerabilidad se encuentra en el **código JavaScript del cliente** (navegador), no en el servidor. El **payload** es procesado y ejecutado directamente por el DOM (**Document Object Model**) del navegador. La aplicación web no tiene control sobre la entrada, ya que no se envía al servidor.

- **Ejemplo:** Un script en la página toma un valor de la URL y lo inyecta directamente en el HTML sin saneamiento. Si el **script** está mal escrito, puede ser vulnerable a la manipulación.
-

3. ¿Cómo testear si una aplicación es vulnerable a XSS?

Para auditar y detectar vulnerabilidades XSS, se utilizan **payloads** específicos.

Payload básico para Cross-Site Scripting

El **payload** más sencillo y común para testear es el siguiente:

```
<script>alert("XSS")</script>
```

- **<script> y </script>:** Estas son las etiquetas HTML que definen un bloque de código JavaScript.
- **alert():** Es una función de JavaScript que muestra una ventana de alerta en el navegador. Es la manera más simple de demostrar que el código se ha ejecutado.
- **"XSS":** Es la cadena de texto que se mostrará en la ventana de alerta.

Sin embargo, este **payload** es muy conocido y muchas aplicaciones tienen filtros para bloquearlo. Por lo tanto, es importante conocer variantes o **payloads** más sofisticados para evadir los filtros de seguridad.

Ejemplos de **payloads** alternativos:

- Sin la etiqueta `<script>`: `<img src=x onerror=alert("XSS")>`
 - `<img src=x>`: Intenta cargar una imagen que no existe.
 - `onerror=alert("XSS")`: Cuando la imagen no se puede cargar, se dispara el evento `onerror`, que ejecuta el código JavaScript dentro del `alert`.
 - **Payloads ofuscados**: Se pueden utilizar codificaciones o técnicas de ofuscación para evadir los filtros que buscan cadenas de texto específicas como "script" o "alert".
-

Conclusión

La mejor forma de prevenir los ataques XSS es a través de una **correcta validación y saneamiento de los datos de entrada**, como se ha visto en el módulo de *Input Validation*. Toda la información recibida debe ser tratada como no confiable y, preferiblemente, se deben utilizar **listas blancas** que solo acepten caracteres y formatos específicos. Implementar estas medidas tanto en el cliente como en el servidor es esencial para proteger a los usuarios y la integridad de la aplicación.

Tipos de XSS: Reflejado, Almacenado y DOM-based

Vamos a desglosar los diferentes tipos de **Cross-Site Scripting** (XSS) y a ver ejemplos prácticos de cómo se explotan en una auditoría web.

1. XSS Reflejado (**Reflected XSS**)

También conocido como XSS no persistente, es el tipo más común y, en general, el de menor impacto. Se le llama "reflejado" porque el código malicioso no se almacena en el servidor, sino que se "refleja" directamente desde la petición HTTP a la respuesta del servidor.

- **Funcionamiento:** Un atacante construye una URL que incluye un **payload** de JavaScript. La URL se envía a la víctima, generalmente a través de un ataque de **ingeniería social** (como un correo electrónico de **phishing**). Cuando la víctima hace clic en el enlace, el navegador envía la petición al servidor, y este, sin validar la entrada, devuelve el **payload** en el código HTML de la respuesta. El navegador de la víctima lo interpreta y lo ejecuta.
- **Riesgos:** Principalmente, compromete la **confidencialidad**. Un atacante puede robar las cookies de sesión del usuario, lo que le permitiría suplantar su identidad. También puede afectar la **integridad de los datos** o la **reputación** de la aplicación.

Procedimiento práctico para la explotación de XSS Reflejado

Vamos a usar el entorno de laboratorio **DVWA (Damn Vulnerable Web Application)** para ilustrarlo. Asumimos que la aplicación está configurada en un nivel de seguridad bajo.

1. **Identificación del punto de entrada:** En el menú de navegación de DVWA, seleccionamos "XSS (Reflected)". Se nos presenta un simple campo de texto que nos pide nuestro nombre. Al ingresar **test**, la página nos devuelve el mensaje "Hello test". Esto indica que la entrada del usuario se está reflejando directamente en la página.
2. **Análisis de la petición:** Podemos usar una herramienta como **Burp Suite** para interceptar la petición. Al ingresar **test** y hacer clic en "Submit", vemos en Burp que se genera una petición GET con el parámetro **name** en la URL. Esto confirma que la entrada va en la URL.

Inyección del **payload:** El objetivo es inyectar código JavaScript para que el navegador lo ejecute. El **payload** más simple para probar la vulnerabilidad es:

JavaScript

```
<script>alert(1)</script>
```

3.
 - **<script> y </script>**: Etiquetas que le dicen al navegador que el contenido entre ellas es JavaScript.
 - **alert(1)**: Es una función de JavaScript que muestra una ventana emergente con el valor **1**. Esta es la forma más sencilla de confirmar que el código se ha ejecutado con éxito.
4. **Ejecución del ataque:** Ingresamos el **payload** en el campo de texto y hacemos clic en "Submit". El navegador inmediatamente muestra una ventana de alerta con el número 1. La vulnerabilidad ha sido confirmada. El **payload** ahora es parte de la URL:

```
http://localhost/vulnerabilities/xss_r/?name=<script>alert(1)</script>#.
```

2. XSS Almacenado (Stored XSS)

Este es el tipo de XSS más peligroso debido a su **naturaleza persistente**. El **payload** malicioso no se refleja, sino que se **almacena en la base de datos** del servidor. El servidor lo entrega a todos los usuarios que visiten la página afectada, lo que multiplica el impacto del ataque.

- **Funcionamiento:** Un atacante inyecta el **payload** en un campo de la aplicación que guarda información (por ejemplo, comentarios de un blog, un libro de visitas, un foro). La aplicación guarda el **payload** en su base de datos sin sanearlo. Cada vez que un usuario legítimo accede a la página que muestra ese contenido, el **payload** se carga desde la base de datos y se ejecuta en su navegador.
- **Riesgos:** El impacto es mucho mayor que el XSS reflejado, ya que **afecta a todos los usuarios que visiten la página vulnerable**. Esto puede llevar a un robo de datos masivo, suplantación de identidad a gran escala y un daño severo a la reputación de la aplicación.

Procedimiento práctico para la explotación de XSS Almacenado

Volvemos a DVWA y esta vez seleccionamos "XSS (Stored)".

1. **Identificación del punto de entrada:** La página muestra un formulario de libro de visitas con dos campos: **Name** y **Message**.
2. **Inyección del **payload**:**
 - Primero, probamos a inyectar el **payload**
`<script>alert("message")</script>` en el campo **Message** y el **Name** lo dejamos como **test**. Al pulsar "Sign Guestbook", la vulnerabilidad se activa y un pop-up con el mensaje "message" aparece. * Ahora, el mensaje **test** y el **payload** están guardados en la base de datos. Si otro usuario o si nosotros mismos volvemos a la página, el pop-up aparecerá de nuevo, demostrando que la inyección es persistente.

3. **Bypassing de validaciones del lado del cliente:** El campo **Name** parece tener una limitación de longitud en el lado del cliente. Si intentamos inyectar el mismo **payload**, no nos deja. Podemos saltarnos esta restricción de dos maneras:
- **Usando Burp Suite:** Interceptamos la petición. Vemos que los datos se envían por **POST** en el cuerpo de la petición. Modificamos la petición para incluir el **payload** completo en el parámetro `txtName=test2<script>alert(name)</script>&mtxMessage=test2&btnSign=Sign+Guestbook`. Al reenviar la petición, el ataque se completa.
 - **Modificando el HTML en el navegador:** Podemos usar las herramientas de desarrollador (**F12**) del navegador para inspeccionar el elemento de entrada del campo **Name**. Buscamos el atributo `maxlength` y cambiamos su valor (por ejemplo, de **10** a **100**). Esto nos permite escribir el **payload** completo en el campo del formulario y enviarlo al servidor.
-

3. XSS Basado en DOM (**DOM-based XSS**)

Este tipo de XSS es diferente porque la vulnerabilidad reside en el **código JavaScript del cliente**, no en el servidor. El **payload** nunca llega al servidor, sino que es ejecutado directamente por el **Document Object Model (DOM)** del navegador.

- **Funcionamiento:** Ocurre cuando un **script** del lado del cliente toma datos de una fuente no confiable (como la URL) y los inyecta dinámicamente en el DOM sin una validación adecuada.
 - **Ejemplo:** Un script lee un parámetro de la URL y lo inserta directamente en el HTML de la página. Si el parámetro contiene un **payload XSS**, este se ejecuta localmente en el navegador de la víctima.

- **Riesgos:** Similares a los del XSS reflejado, ya que requiere que la víctima haga clic en un enlace malicioso. El servidor no tiene visibilidad de este ataque, por lo que los registros de **logs** no mostrarán el **payload**.

Procedimiento práctico para la explotación de XSS Basado en DOM

Usamos un laboratorio de PortSwigger Web Security Academy para este ejemplo.

1. **Análisis de la funcionalidad:** El laboratorio es un blog con una función de búsqueda. Al buscar un término, este se refleja en la página.
2. **Análisis del código fuente:** Inspeccionamos el código JavaScript de la página con las herramientas de desarrollador. Encontramos una parte del código que toma el parámetro de la URL (**location.search**) y lo inserta en el **innerHTML** de un elemento HTML. El uso de **innerHTML** es a menudo una fuente de vulnerabilidad de XSS.

Inyección del **payload:** El **payload** básico `<script>alert(1)</script>` no funciona en este caso. Esto nos indica que hay algún tipo de filtro. Podemos usar un **payload** alternativo que no dependa de la etiqueta `<script>`, por ejemplo:

```
<img src=1 onerror=alert(1)>
```

3.
 - `<img src=1>`: Se intenta cargar una imagen con una URL incorrecta (1).
 - `onerror=alert(1)`: Cuando el navegador no puede cargar la imagen, se dispara el evento **onerror**, que ejecuta el **script** asociado.
 4. **Ejecución del ataque:** Insertamos el **payload** en el campo de búsqueda. El navegador intenta cargar la imagen, falla, y como resultado, se ejecuta el **alert**. El laboratorio confirma que la vulnerabilidad ha sido explotada.
-

Conclusión

El **Cross-Site Scripting** es una amenaza constante en la seguridad web.

Entender los diferentes tipos de XSS (**Reflejado**, **Almacenado** y **Basado en DOM**) y las técnicas para explotarlos es crucial para realizar auditorías efectivas. La clave para prevenir estos ataques, en todos sus tipos, es la **validación y saneamiento estricto de las entradas de datos**, asegurando que ninguna entrada de usuario sea tratada como código ejecutable.

Ejercicio XSS

Introducción al Ejercicio

Este ejercicio está diseñado para que apliques los conocimientos teóricos sobre **Cross-Site Scripting** (XSS) en un entorno práctico. Utilizaremos dos plataformas de laboratorio muy conocidas en el mundo de la ciberseguridad: **DVWA** (**Damn Vulnerable Web Application**) y la **PortSwigger Web Security Academy**.

El objetivo es que explores los diferentes niveles de seguridad implementados en las aplicaciones web y aprendas a evadirlos, entendiendo por qué ciertos **payloads** funcionan y otros no. No te limites a copiar y pegar; el verdadero aprendizaje reside en analizar el porqué detrás de cada **payload**.

1. Ejercicio con DVWA: XSS Reflejado y Almacenado

DVWA es una aplicación web deliberadamente vulnerable que te permite practicar la explotación de diversas vulnerabilidades, incluyendo XSS. Para este ejercicio, te centrarás en los niveles de seguridad **"medio"** y **"alto"** para los ataques de XSS reflejado y almacenado.

Guía para XSS Reflejado (Niveles Medio y Alto)

1. **Configuración:** Asegúrate de que el nivel de seguridad de DVWA esté configurado en "medium".
2. **Análisis:** Ingresa un **payload** básico como `<script>alert(1)</script>`. Es muy probable que no funcione.
3. **Investigación:** Utiliza la funcionalidad de DVWA para **ver el código fuente** de la página. Analiza cómo la aplicación procesa tu entrada. En el nivel medio, es común que se implemente algún filtro, como una **lista negra** (**denylist**) que reemplaza o elimina ciertas palabras clave.

4. **Bypass:** Crea un nuevo **payload** que evada el filtro. Por ejemplo, si la palabra **script** es bloqueada, puedes probar **payloads** que no la usen, como:
 - `<img src=x onerror=alert(1)>`
 - `<body onload=alert(1)>`
 - `<svg onload=alert(1)>`
5. **Nivel alto:** Repite el proceso con el nivel de seguridad "high". Probablemente encontrarás un filtro más robusto o incluso una **lista blanca (allowlist)** que solo acepta entradas muy específicas. El reto aquí es ser más creativo con la ofuscación del **payload** o encontrar un punto de inyección inesperado.
6. **Conclusiones:** Documenta los **payloads** que funcionaron en cada nivel y explica por qué los **payloads** básicos no funcionaron.

Guía para XSS Almacenado (Niveles Medio y Alto)

1. **Configuración:** Cambia el nivel de seguridad de DVWA a "medium" y navega a la sección de XSS Almacenado.
2. **Análisis:** De nuevo, prueba con el **payload** básico `<script>alert(1)</script>`. Observa si se guarda en la base de datos y si se ejecuta al recargar la página.
3. **Investigación:** Revisa el código fuente para entender qué filtros se están aplicando en los campos de **Name** y **Message**. Es posible que haya una validación de longitud, como vimos en el texto anterior, o un filtro que elimine etiquetas HTML o ciertos caracteres especiales.
4. **Evasión:** Diseña un **payload** que eluda las medidas de seguridad. Si el campo **Name** tiene una limitación de longitud, recuerda que puedes usar herramientas como **Burp Suite** o las herramientas de desarrollador del navegador para saltarte esa restricción del lado del cliente y enviar el **payload** completo al servidor.
5. **Nivel alto:** En este nivel, los filtros suelen ser mucho más sofisticados. No basta con cambiar el **payload**; a veces es necesario codificar los caracteres o utilizar combinaciones de **payloads** que

se aprovechen de fallos lógicos en el filtrado. Tu objetivo es encontrar la forma de que el `script` se almacene y ejecute.

2. Ejercicio con PortSwigger Academy: XSS basado en DOM

En esta sección, practicarás la explotación de una vulnerabilidad de `Cross-Site Scripting` (XSS) basada en DOM. Utilizarás la plataforma de laboratorios de la academia de PortSwigger, que te permite simular ataques en un entorno seguro y controlado.

Enunciado del laboratorio:

Este laboratorio contiene una vulnerabilidad de XSS basada en DOM en la funcionalidad de seguimiento de consultas de búsqueda. La vulnerabilidad reside en el uso de la función de JavaScript `document.write`, que escribe datos directamente en la página. Esta función utiliza los datos de `location.search`, que puedes controlar a través de la URL del sitio web. Para resolver el laboratorio, debes realizar un ataque de XSS que ejecute la función `alert`.

Solución propuesta:

1. Accede al laboratorio. En el campo de búsqueda, escribe una cadena de caracteres aleatoria, como `prueba123`.
2. Haz clic derecho en la página y selecciona "Inspeccionar" para abrir las herramientas de desarrollador.
3. Busca tu cadena de caracteres aleatoria en el código HTML. Notarás que ha sido colocada dentro de un atributo `src` de una etiqueta `img`, como en ``.
4. Para inyectar el código malicioso, debes "escapar" o salir de este atributo. Para ello, utiliza la siguiente cadena de caracteres en el campo de búsqueda: `"><svg onload=alert(1)>`
5. Al hacer clic en "Buscar", se ejecutará el código JavaScript inyectado y aparecerá una ventana emergente de `alert`, resolviendo así el laboratorio.

Conclusiones del Ejercicio

Al completar estos ejercicios, habrás experimentado de primera mano las diferencias entre XSS reflejado, almacenado y basado en DOM, y cómo la validación de entrada deficiente permite estos ataques. Habrás aprendido a analizar el código fuente para encontrar debilidades y a construir **payloads** que se salten los filtros de seguridad.

Resolución ejercicio XSS

Vamos a comenzar con el Cross Site Scripting reflejado.

Para ello nos vamos a nuestra máquina virtual, hemos reseteado la base de datos para no tener ahí lo que habíamos metido previamente.

Eso se puede hacer directamente aquí en Setup / Reset DB.

1- Cross Site Scripting reflejado: Nivel medium

Y lo que vamos a hacer va a ser irnos a ese Cross Site Scripting reflejado, primero al nivel de seguridad y le vamos a poner el nivel Medium.

Y nos vamos al Cross Site Scripting reflejado. Hay un formulario en el cual me pide el nombre. Vemos que si pongo Marta me pinte en pantalla “Hello Marta” y si intentamos poniendo nuestro script que funcionó la última vez en nivel Easy:

```
<script>alert(1)</script>
```

Hago submit y me sale un “Hello alert(1)”, parece que me está ignorando los scripts, cosas que pueden estar pasando, que esté dentro de una lista negra, algo que hemos visto que no debería hacerse.

Tenemos la opción de ver el código fuente apretando a View Source justo debajo, una funcionalidad de DVWA para que sea más facil inyectar vulnerabilidades,

entonces vamos a utilizarla para ver el código fuente de justo la vulnerabilidad media. Aquí tenemos el código:

```
<?php
header ("X-XSS-Protection: 0");
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = str_replace( '<script>', "", $_GET[ 'name' ] );
    // Feedback for end user
    echo "<pre>Hello ${name}</pre>";
}
?>
```

Fijaros lo que hace, cuando encuentra la cadena <script>, la cambia por nada, nos la borra.

Así que vamos a ver si en vez de script, que está en minúsculas, lo ponemos con mayúscula:

```
<sCript>alert(1)</script>
```

Submit. Conseguido, me salta el popup del cross site scripting reflejado.

Nos hemos saltado esa lista negra.

Como decía, no es lo suyo utilizar este tipo de listas negras porque los atacantes siempre van a encontrar formas de saltárselo.

De esta manera quedaría explotada la vulnerabilidad de nivel medio del cross site scripting reflejado.

Podríamos utilizar cualquier código JavaScript que no utilice la palabra script y si la utiliza simplemente poniendo mayúsculas, minúsculas nos saltaríamos esa lista negra.

2- Cross Site Scripting reflejado: Nivel hard

Vamos a cambiar ahora a nivel alto, nivel hard, el difícil y nos volvemos al reflejado.

Volvemos a poner el script con la mayúscula y hacemos un submit.

```
<sCript>alert(1)</script>
```

Y ahora me printea "Hello >". Es decir, que no funciona con dicho script, tenemos que encontrar otra manera.

Vamos a ver el código fuente de la página, porque aquí a lo mejor ya tenemos que empezar a pensar que no podemos utilizar script. Tenemos el código:

```
<?php
header ("X-XSS-Protection: 0");
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', "", $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello ${name}</pre>";
}
?>
```

Y aquí efectivamente lo utilizamos como lo utilizamos no debería de funcionar, vamos a probar de ponerlo también en mayúsculas todo, a ver qué es lo que pasaría.

```
<SCRIPT>alert(1)</SCRIPT>
```

Submit y tampoco funciona, ni con las mayúsculas nos conseguimos saltar ya esa medida de seguridad.

Cuando encuentre juntos esos caracteres y lo va a reemplazar por nada, por la línea de código: `$name = preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', "", $_GET[ 'name' ] );`

Así que tenemos que buscar otros payloads que nos puedan servir a la hora de explotar el cross site scripting.

Esto lo podríamos buscar en google como payload cross site scripting y ver qué hay por Internet. Aquí tenemos por ejemplo una lista de payloads. Vamos a ver un poquito que tiene esa lista:

```
"-prompt(8)-"
```

```
"-prompt(8)-"
```

```
";a=prompt,a()"
```

"a=prompt,a()"

'-eval("window['pro%2B'mpt'](8)")-'

'-eval("window['pro%2B'mpt'](8)")-'

"onclick=prompt(8)@x.y"

"onclick=prompt(8)><svg/onload=prompt(8)">@x.y

<image/src/onerror=prompt(8)>

<img/src/onerror=prompt(8)>

<image src onerror=prompt(8)>

<image src=q onerror=prompt(8)>

</scrip/>t>

<svg onload=alert(1)>

"><svg onload=alert(1)//

"onmouseover=alert(1)//

"autofocus/onfocus=alert(1)//

'-alert(1)-'

'-alert(1)-'

\'-alert(1)//

</script><svg onload=alert(1)>

<x contenteditable onblur=alert(1)>lose focus!

<x onclick=alert(1)>copy this!

<x oncopy=alert(1)>copy this!

`<x oncontextmenu=alert(1)>right click this!`

`<x oncut=alert(1)>copy this!`

`<x ondblclick=alert(1)>double click this!`

`<x ondrag=alert(1)>drag this!`

Queremos pues a nivel de intruder, listado de payloads y aquí vemos que empezamos a encontrarnos algunos que utilizan img, algunos que utilizan svg onload que podríamos utilizar y de hecho pues bueno, podemos utilizar este mismamente y vamos a ver si éste funciona porque no lleva el script alert. Lo probamos:

`<svg onload=alert(1)>`

Le damos submit y aquí ya lo tendríamos explotado. Me sale el popup que me indica que hemos conseguido hacer el cross site scripting reflejado.

Simplemente hay que utilizar uno que no utilice el script.

Yo he utilizado el svg onload, pero podríamos por ejemplo utilizar el propio body, que también tiene la opción de poner onload=alert, este también debería de funcionar:

`<body onload=alert(1)>`

Ahí está, funciona perfectamente, diferentes formas de explotar la vulnerabilidad que no impliquen el uso de la palabra script.

Con esto terminada la parte del cross site scripting reflejado, nos vamos de nuevo a cambiar el nivel de seguridad a nivel medio y nos vamos ahora al almacenado.

3- Cross site scripting almacenado: Nivel medium

Recordad que aquí teníamos un formulario para firmar un guestbook con un Name y un Message, y teníamos un problema con la longitud en el formulario de Name, no me cabe el payload entero, y tendríamos que ver cómo podemos hacerlo ahora.

Así que recordamos que a nivel bajo los dos, Name y Message, eran vulnerables y ahora tenemos que averiguar cuál no lo es o cuál sí lo es o si lo son los dos.

Así que vamos a probar de nuevo el script básico de alert `<script>alert(1)</script>` en Message y en Name ponemos test.

Burp aún no está escuchando aquí, probamos antes sin interceptar, lo activamos en FoxyProxy pero no interceptamos, le vamos a decir que queremos firmar el guestbook.

Vemos que no es vulnerable con éste payload script porque no me salta el pop up.

En burp ahora vamos a interceptar para probar de poner el script directamente con el interceptor. Al capturar la petición vemos que nos sale así:

```
txtName=test&mtxMessage=%3Cscript%3Ealert(2)%3C/script%3E&btnSign=Sign+
Guestbook
```

Para inyectar el payload ponemos `<script>alert(1)</script>` al lado de test para que quede así:

```
txtName=test<script>alert(1)</script>&mtxMessage=%3Cscript%3Ealert(2)%3C/scri
pt%3E&btnSign=Sign+Guestbook
```

Vamos a probarlo si funciona, hacemos forward. Aquí no ha hecho nada tampoco. Tenemos que probar otras cosas.

Lo que vamos a hacer ahora va a ser ir capturando esas peticiones, está ahí el intercept y vamos a ir probando de nuevo.

Ponemos test en Name y test en Message, ejecutamos. Vamos a Burp, a la petición interceptada. Nos sale así inicialmente, con test en Name y Message:

```
txtName=test&mtxMessage=test&btnSign=Sign+Guestbook
```

Y ahora aquí lo que vamos a hacer va a ser, en lugar de inyectar el payload de script básico sólo en el name, vamos a probar con las mayúsculas y vamos a inyectarlo en el Name y el Message, por ejemplo con la C mayúscula y la P mayúscula. Quedaría así la inyección:


```
txtName=test<sCriPt>alert(name+medium)</script>&mtxMessage=test<sCriPt>alert(
name+medium)</script>&btnSign=Sign+Guestbook
```

Y vamos a hacer ahora otra vez ese forward.

Aquí ya nos dice que sí, que el nombre es vulnerable en el medium, pero parece que el mensaje no. Porque en el pop up que nos ha aparecido nos salía “name medium”.

Vamos a ver qué pasa en el código fuente.

Tenemos el campo texto de Message y el campo Name, vemos que el código está limpiando el input validation, pero lo hace diferentemente.

Aquí en el Name sí que nos hace el replace de <script> y aquí en el Message nos hace el addslashes, Así que en principio Message no sería vulnerable con el payload que le hemos metido. IMAGEN

El Message también también utiliza el htmlspecialchars, en diferencia con el nombre, los dos utilizan el mysql_real_escape_string que intenta escapar posibles vulnerabilidades, pero aquí en Message hace este htmlspecialchars, lo trata y de esta manera el campo Message no sería vulnerable de ésta manera. Analizemos el código de la imagen:

Análisis de la validación del campo **message**

The code for the **message** input shows three distinct attempts at sanitization:

1. `$message = strip_tags addslashes($message);`
 - **addslashes()**: This function adds backslashes before predefined characters (' , " , \ , NULL). While it's a very basic form of input escaping, it's not a reliable way to prevent XSS. It's more suited for SQL injection, but even for that, modern functions like `mysqli_real_escape_string` are much better and more secure.
 - **strip_tags()**: This function attempts to strip HTML and PHP tags from a string. It's an a **blacklisting** approach, meaning it tries to remove specific malicious content. The problem with `strip_tags()`

is that it can be easily bypassed by using different types of encoding, malformed tags, or non-standard HTML attributes (like `onerror` in `<img>` tags), which we saw in the XSS examples.

2. `$message =`

```
mysqli_real_escape_string($GLOBALS['__mysqli_ston'],  
$message);:
```

- This is the correct function for escaping special characters in SQL queries, which is crucial for preventing **SQL Injection**. However, this function is designed for a database context and is **ineffective against XSS**. It won't stop the execution of JavaScript in the browser.

3. `$message = htmlspecialchars($message);:`

- **htmlspecialchars()**: This is the most effective function shown here for preventing XSS. It converts special characters to their HTML entities. For example, a `<` becomes `<`; and a `>` becomes `>`. When the browser renders the output, it displays `<script>` as literal text, not as a JavaScript command. This function is a **whitelisting** approach, as it ensures the output is treated as plain text, regardless of the input. In a real-world application, this is an excellent method for handling content that should be displayed as text.

The combination of these methods shows the developer's intent to secure the application, but the use of `strip_tags` and `addslashes` indicates a misunderstanding of how XSS attacks work, as they are easily bypassed.

Análisis de la validación del campo `name`

The code for the `name` field reveals a similar, and even more flawed, approach:

1. `$name = str_replace('<script>', '', $name);:`

- This is another example of a **blacklisting** approach. The developer is specifically looking for the string `<script>` and replacing it with nothing.
- This method is extremely easy to bypass. An attacker could use a different case (`<ScRiPt>`), different encodings (`%3Cscript%3E`), or, as you have already seen in the lab, a different HTML tag entirely, such as `<img>` or `<svg>`. A smart attacker would simply use `<img src=x onerror=alert(1)>` and this filter would do absolutely nothing. This demonstrates the critical flaw of blacklisting: you can't possibly think of every single variation an attacker might use.

2. `$name =`

```
mysqli_real_escape_string($GLOBALS['__mysqli_ston'],
$name);:
```

- Just like with the `message` field, this function is only for preventing **SQL Injection** and provides no protection against **XSS**.

In summary, the code shows a developer who is aware of the need for **input validation** but is using flawed, outdated, and insufficient methods. The use of `str_replace` and `strip_tags` is a classic example of a weak **blacklist** approach that an attacker can easily bypass. The correct approach for preventing XSS is **htmlspecialchars** or other robust output encoding methods, which were only partially and inconsistently applied in the first part of the code.

Así que de momento el nivel Medium lo tendríamos explotado.

4- Cross site scripting almacenado: Nivel high

Vamos a ver ahora si lo cambiamos al nivel high, hacemos el submit, nos volvemos al almacenado.

Vamos a hacer ahora una petición con name en Name, y test en Message, y vamos a probar lo que nos pasó anteriormente también.

Submit. Interceptamos con burp, y ahora aquí lo que vamos a probar de inyectar son payloads distintos a script, cómo hemos hecho en reflejado, y los vamos a inyectar de la misma manera, en Name y Message.

Vamos a probar por ejemplo con una imagen, el payload sería:

<img+src=x+onerror=alert("name+high") para Name y
<img+src=x+onerror=alert("message+high") para Message. Quedaría así la petición en burp:

```
txtName=name<img+src=x+onerror=alert("name+high")&mtxMessage=test<img+src=x+onerror=alert("message+high")&btnSign=Sign+Guestbook
```

Le damos a forward, vamos a ver, ha salido el name medium de antes en el nivel medio y no ha salido nada más, no ha funcionado, tendríamos que ver qué es lo que está pasando.

Vamos a hacer ahora una petición con test en Name, y test en Message, y vamos a probar con el payload <body onload=alert(1)>, la petición quedaría así:

```
txtName=test<body onload=alert(1)>&mtxMessage=test<body onload=alert(1)>&btnSign=Sign+Guestbook
```

Nos aparece el popup con el 1, que es el body onload alert 1, significa que hemos explotado la vulnerabilidad con éste payload en nivel high.

Si ahora nos vamos al reflejado y volvemos, nos va a decir el nombre medio y el 1, que es el que hemos utilizado con el body onload igual a alert, que sería este de aquí, es el que hemos probado a modo de resumen, hemos probado este, hemos probado el de img src igual a x, onerror igual a alert, esta sería otra forma, alert y hemos probado este de aquí, el SVG.

Hemos conseguido explotar el nivel medio y alto, tanto el cross site scripting reflejado como el almacenado.

5- DOM-based Cross site scripting

Vámonos ahora con la parte de DOM.

Utilizaremos la plataforma de laboratorios de la academia de PortSwigger, que nos permite simular ataques en un entorno seguro y controlado.

Tenemos este otro laboratorio, Lab DOM in document.write sink using source location.search. Vamos a ver qué es lo que nos pide:

Este laboratorio contiene una vulnerabilidad de XSS basada en DOM en la funcionalidad de seguimiento de consultas de búsqueda. La vulnerabilidad reside en el uso de la función de JavaScript `document.write`, que escribe datos directamente en la página. Esta función utiliza los datos de `location.search`, que puedes controlar a través de la URL del sitio web. Para resolver el laboratorio, debes realizar un ataque de XSS que ejecute la función `alert`.

Contiene un DOM Based Cross Site Scripting en la query de la funcionalidad del traqueo. Utiliza javascript document.write la función y escribe datos fuera de la página. El document.write se llama desde location.search, que podemos controlar utilizando la URL y tenemos que resolver este laboratorio llamando a una función de alert.

Así que abrimos el laboratorio, vamos a ver cómo funciona, es un blog con un formulario de búsqueda que tenemos que explotar.

Vamos a ponerle aquí por ejemplo "history" que aparece en el primer blog, nos dice que hay un resultado y vamos a ver cómo está funcionando la aplicación. Nos vamos a las tools de desarrollador y vamos a buscar history, que tiene que aparecer varias veces, la primera en el result, la segunda dentro de una imagen, fijaros, dentro de un script hace una imagen, resource image y hace los términos de búsqueda y nos llega al history y luego ya pues en el campo donde está aquí el history serán los tres sitios donde nos aparece.

Vamos a intentar crear nuestro alert, pero vamos a poner el payload básico de alert 1, que sería:

```
history<script>alert(1)</script>
```

Vamos a ver qué pasa. Damos clic en search, aquí ya no nos aparece, vamos a buscar en las tools de desarrollador, filtramos por history, nos queda dentro una

imagen, esta imagen lo que vamos a hacer va a ser cerrarla para que se ejecute nuestro alert. IMAGEN2

Entonces podemos cerrar la imagen, vemos que nos lo pone con la comilla, pues vamos a cerrarlo así como con la doble comilla y a ver si esto lo ejecuta:

```
history"><script>alert(1)</script>
```

Le damos a search y esto lo ejecuta, me sale el pop up con el 1 del alert, vulnerabilidad explotada.

Vamos a explicarlo: vamos a volver a las tools de desarrollador y al buscar history en el formulario vemos que encuentra un resultado, queremos saber qué encuentra, y en las tools vemos que es una imagen. IMAGEN 3

```
<section>
  <h1>0 search results for 'history"><script>alert(1)</script>'</h1>
  <hr>
</section>
```

```
<section class="search">...</section>
```

```
<script>
  function trackSearch(query) { document.write(''); }
  var query = (new URLSearchParams(window.location.search)).get('search');
  if(query) {
    trackSearch(query);
  }
</script>
```

```

```

```
<script>alert(1)</script>
```

Breakdown of the Attack

The image illustrates a classic **reflected XSS attack**. Here's what's happening:

1. **User Input:** The attacker enters a malicious string into the search bar:

```
history"><script>alert(1)</script>.
```

2. **Lack of Sanitization:** The server or application takes this user input and directly inserts it into the HTML of the page without properly sanitizing or encoding it.
3. **Code Execution:** The browser renders the page and sees the `<h1>` tag with the injected content. The `<script>alert(1)</script>` part is no longer just a string; it's interpreted as a real JavaScript command.
4. **Malicious Action:** The `alert(1)` script is executed, causing a pop-up box to appear. While this is a harmless example, a real attacker could use this vulnerability to steal session cookies, redirect users to malicious sites, or deface the website.

The line `document.write('');` is particularly vulnerable because it uses `document.write()` to directly embed unsanitized user input (`query`) into the page's HTML. This is what allows the attacker to break out of the `<img>` tag's `src` attribute and inject their own script.

a poner a buscar el history, Fijaros, aquí tenemos una imagen que es la que se ejecuta aquí arriba, esta parte del img src nos mete la búsqueda dentro de esta imagen history o lo que pongamos, como esto es lo de antes, pues el history, el alert, etcétera.

```

```

Como quiero cerrar este tag de la imagen y yo estoy justo aquí en `=history`, lo que le voy a decir es que me cierre, por eso pongo las dos comillas, luego le pongo el cierre porque con esto cierra toda la imagen y ahora yo abro lo que quiera.

Exactamente así se inyecta el payload, cerrando la imagen asociada a history y luego le pongo el payload, y cierro la imagen poniendo en el search query history"> y el payload:

```
history"><script>alert(1)</script>
```

Funcionaría también con el payload de `body onload`, y sin buscar la imagen de history específicamente también:

"><body onload=alert(1)>

Hacemos ahora el search y funciona, nos ha vuelto a sacar la ejecución de nuestro payload. Ha cerrado el tag de la imagen cualquiera, y luego nos ha dejado inyectar el payload body onload.

Con esto tendríamos terminados los ejercicios tanto de Cross site scripting reflejado, del almacenado y del DOM-based Cross site scripting.

Vamos a desglosar la resolución de los ejercicios de XSS, explicando paso a paso cómo se explotan las vulnerabilidades en los diferentes niveles de seguridad.

1. XSS Reflejado: Nivel Medio

En el nivel medio de DVWA, la aplicación ha añadido un filtro básico para prevenir ataques XSS.

Análisis de la vulnerabilidad: Al intentar inyectar el `payload` `<script>alert(1)</script>`, la página lo ignora y muestra `Hello alert(1)`. Esto sugiere que el servidor está filtrando la palabra "script". Si revisas el código fuente, verás la razón:

```
<?php
// ...

$name = str_replace( '<script>', '', $_GET[ 'name' ] );

// ...

?>
```


La función `str_replace` busca la cadena exacta `<script>` y la sustituye por una cadena vacía. Esta es una medida de seguridad débil y un ejemplo clásico del enfoque de **lista negra**.

Solución y Explotación: Para evadir este filtro, simplemente necesitas escribir la palabra "script" con una combinación de mayúsculas y minúsculas. El `str_replace` es sensible a las mayúsculas, por lo que no detectará la cadena.

- **Payload de ataque:** `<sCript>alert(1)</script>`
 - **Resultado:** El servidor no encuentra el `string <script>` en minúsculas y lo deja pasar. El navegador lo interpreta como una etiqueta válida y ejecuta el `alert`.
-

2. XSS Reflejado: Nivel Alto

En este nivel, la defensa es más robusta y requiere un enfoque diferente.

Análisis de la vulnerabilidad: Al intentar usar el `payload` del nivel medio (`<sCript>alert(1)</script>`), la inyección falla. Al revisar el código fuente, encontramos un nuevo filtro más complejo:

```
<?php
// ...

$name = preg_replace( '/<(.*?)s(.*?)c(.*?)i(.*?)p(.*?)t/i', '', $_GET[ 'name' ] );

// ...

?>
```

Este código utiliza una **expresión regular** (`preg_replace`) con la bandera `i` para realizar una búsqueda **insensible a mayúsculas y minúsculas**. Esto significa que

buscará y eliminará cualquier variación de la palabra "script", sin importar si está en mayúsculas o minúsculas.

Solución y Explotación: Dado que la palabra "script" está completamente bloqueada, debemos usar un **payload** que no la contenga. Podemos utilizar etiquetas HTML que soporten eventos JavaScript.

- **Payloads de ejemplo:**
 - `<svg onload=alert(1)>`
 - `<body onload=alert(1)>`
 - **Mecanismo de ataque:** Estas etiquetas tienen atributos de evento como **onload** que se activan cuando la página o el elemento se cargan. El **payload** `<svg onload=alert(1)>` le dice al navegador que, cuando la etiqueta `<svg>` se cargue, ejecute la función `alert(1)`. Dado que el servidor no tiene un filtro para "svg" o "onload", la inyección tiene éxito.
-

3. XSS Almacenado: Nivel Medio

En el nivel medio de XSS almacenado, se valida de forma diferente el campo **Name** y el campo **Message**.

Análisis del código fuente: Revisando el código de la página, podemos ver que el campo **name** utiliza un `str_replace` similar al XSS reflejado, que solo filtra la cadena `<script>`. El campo **message**, en cambio, utiliza `addslashes` y `htmlspecialchars`. Este último es un filtro mucho más efectivo, ya que convierte los caracteres `<` y `>` a sus equivalentes HTML `<` y `>`, neutralizando el código.

Solución y Explotación:

- **Campo Name:** Dado que el filtro es idéntico al del XSS reflejado de nivel medio, podemos evadirlo usando la misma técnica. Podemos usar un

`payload` como `<script>alert(1)</script>` o cualquier otro que no contenga la palabra `script` en minúsculas.

- **Campo Message:** Debido al uso de `htmlspecialchars`, el campo `Message` no es vulnerable a inyecciones de `scripts` de esta manera, ya que el código se mostrará como texto plano.
-

4. XSS Almacenado: Nivel Alto

En el nivel más difícil, los filtros son más completos y el `payload` debe ser aún más creativo.

Análisis del código fuente: En este nivel, ambos campos (`name` y `message`) utilizan funciones de saneamiento más robustas, como `htmlspecialchars` o expresiones regulares más complejas, para prevenir la inyección de `scripts`. Los `payloads` que funcionan en niveles inferiores no funcionarán aquí.

Solución y Explotación: La clave es encontrar un `payload` que no sea detectado por el filtro y que se ejecute en el navegador.

- **Payload de ejemplo:** En este nivel, un `payload` como `<body onload=alert(1)>` puede funcionar porque el filtro podría estar centrado en etiquetas comunes de inyección (`script`, `img`) pero no en todas las posibles etiquetas HTML que soportan eventos.
 - **Mecanismo de ataque:** Se inyecta el `payload` en el campo, se guarda en la base de datos y, cuando la página se vuelve a cargar, el navegador interpreta la etiqueta `<body>` con su evento `onload` y ejecuta el `alert`. Esto demuestra que la validación, aunque robusta, aún no es perfecta.
-

5. XSS basado en DOM

Este ejercicio se centra en la vulnerabilidad en el código JavaScript del cliente.

Análisis de la vulnerabilidad: La página tiene una función de búsqueda que toma la entrada del usuario de la URL (`location.search`) y la inyecta directamente en la página usando `document.write()`.

- **Código vulnerable:** `document.write('');`
- **Flujo del ataque:**
 1. El usuario busca un término.
 2. La entrada se añade a la URL.
 3. El `script` de la página lee la URL y la escribe dinámicamente en el HTML.
 4. El `payload` se ejecuta en el navegador.

Solución y Explotación: El `script` vulnerable inserta la entrada dentro de la etiqueta `<img>`. Para ejecutar nuestro código, debemos "escapar" o salir de la etiqueta `<img>` y crear una nueva.

- **Payload de ataque:** `history"><script>alert(1)</script>`
- **Mecanismo de ataque:**
 1. La parte `history"` cierra el atributo `src` de la imagen.
 2. El símbolo `>` cierra la etiqueta `<img>`.
 3. A partir de ahí, el `payload <script>alert(1)</script>` se interpreta como código JavaScript válido.

Con esta inyección, el navegador renderiza el código correctamente, ejecuta el `alert`, y la vulnerabilidad de XSS basada en DOM queda expuesta.